Question 1 [10 Marks]

A designated sub-module of the Library System is tasked with capturing and maintaining records of discussion room reservations. This system allows users to reserve the discussion rooms within the library for collaborative activities. The basic types in this system are given as follows:

[DIS_ROOM]	the set of all the discussion room
[USERS]	the set of all user

The state of this Library System is represented by the following schema:

```
┌── LIBRARY_SYS ──────────────────────
│ bookedTo: DIS_ROOM ⇸ USERS
└─────────────────────────────────────
```

One of the main operation in this system is to accept a booking which is illustrate on below shema:

```
┌── AcceptBooking ────────────────────   line 1
│ Δ LIBRARY_SYS                          line 2
│ users? : USERS                         line 3
│ d_room? : DIS_ROOM                     line 4
├─────────────────────────────────────
│ d_room? ∉ dom bookedTo                 line 5
│ bookedTo' = bookedTo ∪ { d_room? ↦ users?}   line 6
└─────────────────────────────────────
```

tested = Variable

Explain the meaning and purpose of each line in the above operation schema. [2m/line]

example :

Line 1 : Name of the schema

head, tail.
ke head buang head

Question 2 [10 Marks]

Let α, β, λ: seq *Char* be sequences of characters defined like below:

(last) front
last buang last

$$\alpha == \langle M, D, K \rangle$$
$$\beta == \langle C, J \rangle$$
$$\lambda == \langle P \rangle$$

What are the following sequences?

⟨C,J,M,D,K⟩ ^ ⟨P,M,D,K⟩

a) $(\beta \frown \alpha) \frown (\lambda \frown \alpha)$ [2m]

⟨C,J,P,M,D,K⟩

head ⟨C,J,M,D⟩

b) *head* (*front* $(\beta \frown \alpha)$) [2m]

⟨C⟩

⟨C,J,M,D⟩

c) *last* (*front* $(\beta \frown \alpha)$) [2m]

⟨D⟩

⟨C,J,M,D⟩

d) *tail* (*front* $(\beta \frown \alpha)$) [2m]

~~⟨M⟩~~ ⟨J,M,D⟩

⟨M,D⟩ ^ ⟨C,J⟩

e) *squash* $(1 \ldots 2 \lhd \alpha) \frown$ *squash* $(1 \ldots 2 \lhd \beta)$ [2m]

⟨M,D,C,J⟩

10

CamScanner

Question 3 [15 Marks]

A computerized class manager's assistant is required to keep track of students enrolled on a class, and to record which of them have done the test. When a student applies for a class, he or she will be enrolled on it unless it is full. Such a student presumed not to have done the test. When a student completes the test, the fact is to be recorded. Students may leave a class even if they have not done the test but only the students who have done the test are entitled to a completion certificate. The operations required are as follows:

Operation	Processing
[Enrol]	Enrol a student on a class, or issues a warning if the class is full, or if the student is already enrolled
[Test]	Records that a student has done the test, or warns if the student is not enrolled, or has already done test
[Leave]	Removes a student from the class with an indication of whether the student is entitled to a completion certificate. Only students who have done the test are entitled to a certificate. If the student is not enrolled, a warning is given

c? enrolled.

c? ∈ enrolled
enrolled' = enrolled \ {c?}

Then this informal specifications are transform into formal specification.

[Student]

Maxsize : ℕ
Maxsize = 40

Response ::= success | notenrolled | nocert | cert | alreadyenrolled | alreadytested | noroom

Class
enrolled, tested : ℙ Student
#enrolled ≤ Maxsize Tested ⊆ enrolled

enrolled : ℙ student
tested : ℙ student

u? : university

a) Complete all the given schemas

c? : CLASSROOM
l? : LECTURER

u? ∈ participating
participating' = participating ∪ {u?}

c? ∉ dom ~~class~~ bookedTo
bookedTo' = bookedTo ⊕ {c? → l?}

CSD33403 FORMAL METHODS /FIK/ SEMESTER II SESSION 2024/2025

s? : Student

Enrol

ΔClass

(a) e? : ~~Enrol~~ [1m]

r! : Response

(b) e? ∉ enrolled [2m]

enrolled < Maxsize

(c) ~~e?~~ enrolled' = enrolled ∪ {e?} [2m] // to enrolled the student

tested' = tested

r! = success

~~a) s? = Student~~

~~b) s? ∈ enrolled~~

Test

(d) t? : ~~tested~~ Test [1m]

s? : Student

r! : Response

s? ∈ enrolled

(e) t? ∈ tested [2m]

(f) tested' = tested ⊕ {s?} [2m] // to record the student has done the test

enrolled' = enrolled

r! = success

ΔClass

LeaveOk

ΔClass

(g) ~~t? : Leave~~ [1m]

r! : Response

(h) ~~t? ∈ enrolled~~ [2m]

(i) ~~enrolled' = enrolled \ {t?}~~ [2m] // to delete the student

((s? ∈ tested ∧ tested' = tested ⊕ {s?} ∧ r! = cert)

∨ (s? ∈ tested ∧ tested' = tested ∧ r! = nocert))

~~r? = Leave~~

g) ~~s? = Student~~

s? = Student

h) s? ∈ enrolled

i) ~~s?~~ enrolled' = enrolled \ {s?}

~~s?~~ student' = student \ {s?}

Student

teave